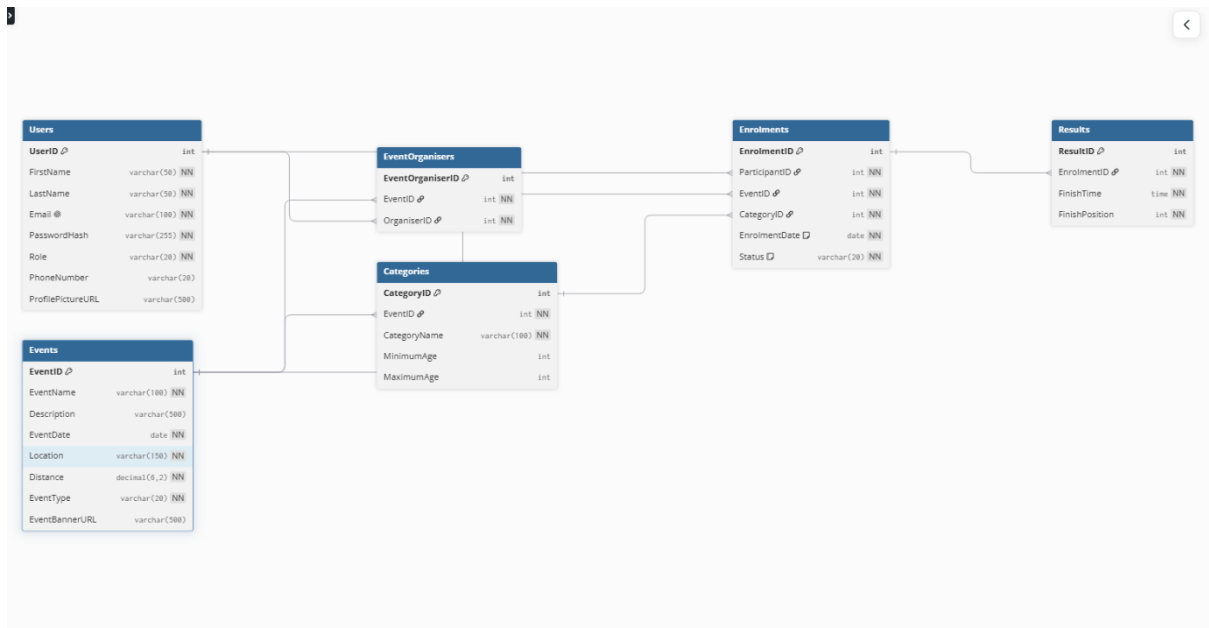


SECTION A



SECTION B

RaceDay – Section B: API Endpoint Plan

Programming 2B (PROG6212) | Part 1 – System Planning

The endpoint plan covers Authentication, User Profile, Events, Categories, Event Enrolments and Results. It is intended to be used as the specification for the Part 2 RESTful API.

1. Authentication

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Creates a new Participant account.	None	{ "firstName", "lastName", "email", "password", "phoneNumber" }	201 Created - account created. 400 Bad Request - invalid information. 409 Conflict - email already exists.
POST	/api/auth/login	Authenticates a user and returns their user ID and role.	None	{ "email", "password" }	200 OK - login successful. 401 Unauthorized - incorrect email or password.

2. User Profile

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/users/profile	Gets the profile information of the logged-in user.	Any	None	200 OK - profile returned. 401 Unauthorized - not logged in. 404 Not Found - profile does not exist.
PUT	/api/users/profile	Updates the logged-in user's personal information.	Any	{ "firstName", "lastName", "phoneNumber", "profilePictureURL" }	200 OK - profile updated. 400 Bad Request - invalid information. 401 Unauthorized - not logged in.

3. Events

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events	Returns a list of upcoming events.	None	None	200 OK - list of events returned.
GET	/api/events/{id}	Returns details for one event, including its categories.	None	None	200 OK - event returned. 404 Not Found - event does not exist.
POST	/api/events	Creates a new running, walking, or cycling event.	Organiser	{ "eventName", "description", "eventDate", "location", "distance", "eventType", "eventBannerURL" }	201 Created - event created. 400 Bad Request - invalid information. 401 Unauthorized - not logged in. 403 Forbidden - not an Organiser.
PUT	/api/events/{id}	Updates an existing event.	Organiser	{ "eventName", "description", "eventDate", "location", "distance", "eventType", "eventBannerURL" }	200 OK - event updated. 404 Not Found - event does not exist. 403 Forbidden - not an Organiser.
DELETE	/api/events/{id}	Deletes an event.	Organiser	None	204 No Content - event deleted. 404 Not Found - event does not exist. 403 Forbidden - not an Organiser.

4. Categories

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events(eventId)/categories	Gets all categories for a specific event.	None	None	200 OK - categories returned. 404 Not Found - event does not exist.

5. Event Enrolments

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/events(eventId)/enrolments	Enrols the logged-in Participant in an event using a selected category.	Participant	{ "categoryId" }	201 Created - enrolment created. 400 Bad Request - invalid category. 401 Unauthorized - not logged in. 403 Forbidden - not a Participant. 409 Conflict - already enrolled.
GET	/api/enrolments/my	Returns all events entered by the logged-in Participant.	Participant	None	200 OK - enrolments returned. 401 Unauthorized - not logged in.
GET	/api/events(eventId)/enrolments	Returns all Participants enrolled in a specific event.	Organiser	None	200 OK - enrolments returned. 404 Not Found - event does not exist. 403 Forbidden - not an Organiser.
GET	/api/enrolments/{id}	Gets details of a specific enrolment.	Any	None	200 OK - enrolment returned. 404 Not Found - enrolment does not exist. 401 Unauthorized - not logged in.

6. Results

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/enrolments(enrolmentId)/result	Records a Participant's finish time and finishing position.	Organiser	{ "finishTime", "finishPosition" }	201 Created - result recorded. 400 Bad Request - invalid result. 404 Not Found - enrolment does not exist. 403 Forbidden - not an Organiser.
PUT	/api/results/{id}	Updates an existing race result.	Organiser	{ "finishTime", "finishPosition" }	200 OK - result updated. 404 Not Found - result does not exist. 403 Forbidden - not an Organiser.
GET	/api/results/my	Returns the logged-in Participant's race history.	Participant	None	200 OK - personal results returned. 401 Unauthorized - not logged in.
GET	/api/events(eventId)/results	Returns results for all Participants in an event.	Organiser	None	200 OK - event results returned. 404 Not Found - event does not exist. 403 Forbidden - not an Organiser.

SECTION C

CREATE DATABASE RACE_DAY;

USE RACE_DAY;

CREATE TABLE USERS

(

UserID INT PRIMARY KEY,

FirstName VARCHAR(50) NOT NULL,

LastName VARCHAR(50) NOT NULL,

Email VARCHAR(100) NOT NULL UNIQUE,

```
Password VARCHAR(100) NOT NULL,  
Role VARCHAR(20) NOT NULL  
);
```

```
-----  
-- STEP 4: CREATE THE EVENTS TABLE  
-----
```

```
CREATE TABLE EVENTS  
(  
    EventID INT PRIMARY KEY,  
    EventName VARCHAR(100) NOT NULL,  
    EventDescription VARCHAR(255),  
    EventDate DATE NOT NULL,  
    Location VARCHAR(100) NOT NULL,  
    Distance DECIMAL(5,2) NOT NULL,  
    EventType VARCHAR(20) NOT NULL,  
    OrganiserID INT NOT NULL,  
  
    FOREIGN KEY (OrganiserID)  
    REFERENCES USERS(UserID)  
);
```

```
-----  
-- STEP 5: CREATE THE CATEGORIES TABLE  
-----
```

```
CREATE TABLE CATEGORIES
(
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(50) NOT NULL,
    EventID INT NOT NULL,

    FOREIGN KEY (EventID)
    REFERENCES EVENTS(EventID)
);
```

-- STEP 6: CREATE THE ENROLMENTS TABLE

```
CREATE TABLE ENROLMENTS
(
    EnrolmentID INT PRIMARY KEY,
    ParticipantID INT NOT NULL,
    EventID INT NOT NULL,
    CategoryID INT NOT NULL,
    EnrolmentStatus VARCHAR(20) DEFAULT 'Confirmed',

    FOREIGN KEY (ParticipantID)
    REFERENCES USERS(UserID),

    FOREIGN KEY (EventID)
```

```
REFERENCES EVENTS(EventID),

FOREIGN KEY (CategoryID)
REFERENCES CATEGORIES(CategoryID)
);
```

```
-- STEP 7: CREATE THE RESULTS TABLE
```

```
CREATE TABLE RESULTS
(
    ResultID INT PRIMARY KEY,
    EnrolmentID INT NOT NULL,
    FinishTime TIME NOT NULL,
    FinishingPosition INT NOT NULL,

    FOREIGN KEY (EnrolmentID)
    REFERENCES ENROLMENTS(EnrolmentID)
);
```

```
-- STEP 8: CREATE THE EVENT IMAGES TABLE
```

```
CREATE TABLE EVENT_IMAGES
```

```
(  
    ImageID INT PRIMARY KEY,  
    EventID INT NOT NULL,  
    ImageURL VARCHAR(255),  
  
    FOREIGN KEY (EventID)  
    REFERENCES EVENTS(EventID)  
);
```

```
-----  
-- STEP 9: INSERT USERS  
-----
```

```
INSERT INTO USERS  
VALUES  
(1, 'John', 'Mokoena', 'john@gmail.com', 'John123', 'Organiser');
```

```
INSERT INTO USERS  
VALUES  
(2, 'Thabo', 'Nkosi', 'thabo@gmail.com', 'Thabo123', 'Organiser');
```

```
INSERT INTO USERS  
VALUES
```

```
(3, 'Mphiwa', 'Bucibo', 'mphiwa@gmail.com', 'Mphiwa123', 'Participant');
```

```
INSERT INTO USERS
```

```
VALUES
```

```
(4, 'Sihle', 'Dlamini', 'sihle@gmail.com', 'Sihle123', 'Participant');
```

```
-----
```

```
-- STEP 10: INSERT EVENTS
```

```
-----
```

```
INSERT INTO EVENTS
```

```
VALUES
```

```
(1, 'Soweto Marathon',  
'Annual running event in Soweto',  
'2026-11-01',  
'Soweto',  
42.20,  
'Run',  
1);
```

```
INSERT INTO EVENTS
```

```
VALUES
```

```
(2, 'Johannesburg Cycle Tour',  
'Cycling event around Johannesburg',  
'2026-10-15',  
'Johannesburg',
```

50.00,

'Cycle',

2);

INSERT INTO EVENTS

VALUES

(3, 'Cape Town Fun Walk',

'Community walking event',

'2026-12-05',

'Cape Town',

10.00,

'Walk',

1);

-- STEP 11: INSERT CATEGORIES

INSERT INTO CATEGORIES

VALUES

(1, 'Senior', 1);

INSERT INTO CATEGORIES

VALUES

(2, 'Under 20', 1);

INSERT INTO CATEGORIES

VALUES

(3, 'Open', 2);

INSERT INTO CATEGORIES

VALUES

(4, 'Veteran', 2);

INSERT INTO CATEGORIES

VALUES

(5, 'Adult', 3);

INSERT INTO CATEGORIES

VALUES

(6, 'Junior', 3);

-- STEP 12: INSERT ENROLMENTS

INSERT INTO ENROLMENTS

VALUES

(1, 3, 1, 1, 'Confirmed');

INSERT INTO ENROLMENTS

VALUES

(2, 4, 1, 2, 'Confirmed');

INSERT INTO ENROLMENTS

VALUES

(3, 3, 2, 3, 'Confirmed');

INSERT INTO ENROLMENTS

VALUES

(4, 4, 3, 5, 'Confirmed');

-- STEP 13: INSERT RESULTS

INSERT INTO RESULTS

VALUES

(1, 1, '03:45:20', 47);

INSERT INTO RESULTS

VALUES

```
(2, 2, '04:10:15', 85);
```

```
-----  
-- STEP 14: INSERT EVENT IMAGES  
-----
```

```
INSERT INTO EVENT_IMAGES
```

```
VALUES
```

```
(1, 1, 'soweto-marathon.jpg'),
```

```
(2, 2, 'johannesburg-cycle.jpg'),
```

```
(3, 3, 'cape-town-walk.jpg');
```

```
-----  
-- STEP 15: VIEW THE TABLES  
-----
```

```
SELECT * FROM USERS;
```

```
SELECT * FROM EVENTS;
```

```
SELECT * FROM CATEGORIES;
```

```
SELECT * FROM ENROLMENTS;
```

```
SELECT * FROM RESULTS;
```

```
SELECT * FROM EVENT_IMAGES;
```

```
SELECT
```

```
    USERS.FirstName,
```

```
    USERS.LastName,
```

```
    EVENTS.EventName,
```

```
    EVENTS.EventDate,
```

```
    EVENTS.Location
```

```
FROM USERS
```

```
INNER JOIN EVENTS
```

```
    ON USERS.UserID = EVENTS.OrganiserID;
```

```
SELECT
```

```
    USERS.FirstName,
```

```
    USERS.LastName,
```

```
    EVENTS.EventName,
```

```
    ENROLMENTS.EnrolmentStatus
```

```
FROM ENROLMENTS
```

```
INNER JOIN USERS
```

```
    ON ENROLMENTS.ParticipantID = USERS.UserID
```

```
INNER JOIN EVENTS
```

```
    ON ENROLMENTS.EventID = EVENTS.EventID;
```

```
SELECT
```

```
    USERS.FirstName,
```

```
    USERS.LastName,
```

```
EVENTS.EventName,
CATEGORIES.CategoryName,
ENROLMENTS.EnrolmentStatus
FROM ENROLMENTS
INNER JOIN USERS
    ON ENROLMENTS.ParticipantID = USERS.UserID
INNER JOIN EVENTS
    ON ENROLMENTS.EventID = EVENTS.EventID
INNER JOIN CATEGORIES
    ON ENROLMENTS.CategoryID = CATEGORIES.CategoryID;

SELECT
ENROLMENTS.EnrolmentID,
USERS.FirstName,
USERS.LastName,
RESULTS.FinishTime,
RESULTS.FinishingPosition
FROM RESULTS
INNER JOIN ENROLMENTS
    ON RESULTS.EnrolmentID = ENROLMENTS.EnrolmentID
INNER JOIN USERS
    ON ENROLMENTS.ParticipantID = USERS.UserID;
```